

Relatório técnico - Servidor HTTP/1.1 sobre sockets TCP

Laboratório de Redes de Computadores - Trabalho 1 Grupo: 7 Implementação: Python 3.9+ e biblioteca padrão **Estado:** código completo; campos experimentais aguardam captura entre máquinas distintas.

Este documento explica o código e cobre os dez pontos exigidos no enunciado. Valores marcados **PENDENTE** dependem de Wireshark e ping na rede do laboratório. Resultados de loopback não foram apresentados como medições.

1. Objetivo e ambiente

O servidor implementa HTTP/1.1 diretamente sobre TCP. Ele recebe conexões de outras máquinas, interpreta bytes da requisição, lê arquivos sob uma raiz configurada e constrói manualmente a resposta. A Parte 2 acrescenta persistência: dez requisições podem usar o mesmo socket. A comparação C1/C2 mede quanto o custo de abrir conexões TCP afeta pacotes, bytes e tempo.

Execução: `python3 server.py --port 8080 --root ./www` (no Windows, `py -3`). O programa faz bind em 0.0.0.0, porta acima de 1024. Cliente e servidor devem estar em máquinas distintas para a avaliação. O filtro Wireshark é `tcp.port == 8080 && ip.addr == IP_CLIENTE` na interface real.

Ambiente da medição: servidor/OS/Python: **PENDENTE**; cliente/OS/Python: **PENDENTE**; IPs: **PENDENTE**; porta: **PENDENTE**; interface: **PENDENTE**; data: **PENDENTE**.

2. Arquitetura e explicação do código

Componente	Responsabilidade e funções principais
server.py	main valida CLI; HTTPServer.serve_forever executa socket, bind, listen, accept; _serve_client atende uma conexão; _response_for escolhe status e arquivo; shutdown fecha o socket principal.
http_protocol.py	receive_request_head acumula bytes até \r\n\r\n; parse_request interpreta linha e cabeçalhos; build_response serializa status, cabeçalhos e corpo; SERVER_ID é Grupo7.
file_service.py	percent_decode_path decodifica o caminho; resolve_target impede fuga da raiz e escolhe index.html; content_type_for mapeia extensões.
tools/measure_connections.py	Cliente de medição: C1 cria dez sockets; C2 cria um socket e realiza dez GETs sequenciais. Lê cada resposta por Content-Length.
tools/slow_client.py	Divide uma requisição em duas partes com intervalo para demonstrar que uma conexão lenta não bloqueia outra.
tools/idle_client.py	Faz um GET, mantém o socket aberto e mede o fechamento por ociosidade.

Fluxo de uma conexão: accept entrega um socket à thread; recv preenche o buffer; a linha vazia CRLF encerra os cabeçalhos; o parser cria HTTPRequest; o serviço de arquivos resolve o alvo; build_response monta a resposta; sendall envia todos os bytes; a thread volta a ler o mesmo socket, ou o fecha.

O TCP oferece um **fluxo de bytes**, não mensagens HTTP prontas. Um recv pode terminar no meio do cabeçalho ou conter várias requisições. Por isso, receive_request_head(client, pending) acumula até \r\n\r\n e devolve (head, leftover). _serve_client passa leftover à próxima iteração. O limite de 64 KiB de cabeçalhos evita crescimento ilimitado do buffer.

parse_request separa método, request-target e versão; aceita apenas a versão HTTP/1.1, exatamente um Host não vazio e cabeçalhos no formato nome: valor. Nomes são comparados sem distinguir maiúsculas. Respostas malformadas recebem 400 e fechamento. build_response usa CRLF, inclui sempre Date no formato GMT, Server: Grupo7, Content-Length, Content-Type e Connection. Content-Length conta **bytes**, inclusive no HEAD e nos erros. HEAD omite somente o corpo. O código não usa http.server, Flask ou outro módulo de servidor HTTP.

Detalhe do controle de fluxo em server.py: valid_port rejeita portas fora de 1025–65535 e existing_directory rejeita uma raiz inexistente antes de abrir o socket. serve_forever usa SO_REUSEADDR, associa host:port, inicia listen, espera em accept e cria uma thread nomeada para cada cliente. _serve_client configura timeout de 5 s, mantém o buffer pending entre as iterações e chama _response_for para aplicar os métodos e códigos. _error_response padroniza o HTML de erro; _client_requests_close procura o token close na lista de valores de Connection; e _has_unsupported_request_body impede que um corpo seja confundido com a próxima requisição. shutdown sinaliza a parada, fecha o socket de escuta e espera as threads dentro de um limite.

Detalhe do protocolo em http_protocol.py: receive_request_head separa o primeiro bloco de cabeçalhos da sobra sem assumir que um recv equivale a uma requisição. ClientDisconnected distingue EOF sem nova mensagem de um cabeçalho truncado, que gera BadRequestError. parse_request valida método como token,

alvo na forma /caminho, versão HTTP/1.1 e sintaxe de cada cabeçalho; os nomes são normalizados para minúsculas. build_response escolhe a frase do status, serializa em CRLF e calcula Content-Length a partir dos bytes do corpo. A constante SERVER_ID concentra o identificador Grupo7.

Detalhe dos arquivos em file_service.py: percent_decode_path converte pares %HH em bytes UTF-8 e rejeita codificação inválida ou NUL. resolve_target remove a query, trata barras invertidas como separadores, rejeita . . , resolve o caminho real e confirma que ele está dentro da raiz, inclusive depois de seguir links simbólicos; diretórios procuram index.html. content_type_for faz a escolha explícita do MIME pela extensão. O arquivo é lido com read_bytes, preservando também recursos binários.

3. Concorrência e justificativa

O laço de accept cria uma thread por conexão. Cada thread bloqueia apenas no seu próprio recv/sendall; uma requisição lenta não impede outra de ser aceita. A coleção de threads é protegida por Lock para permitir encerramento seguro. Essa escolha é simples de explicar e suficiente para o volume pequeno do laboratório. Ela consome uma thread por cliente; para grande escala seria melhor limitar workers ou usar I/O não bloqueante.

Evidência exigida: cliente B envia cabeçalho parcial por 3 s usando tools/slow_client.py; cliente C faz GET durante esse intervalo. Registrar IPs, horários, respostas e log/captura: **PENDENTE**. O teste automatizado equivalente passou em loopback, mas a evidência final deve usar duas máquinas clientes distintas.

4. Segurança do diretório raiz

resolve_target separa a query, decodifica percent-encoding, normaliza as barras e rejeita segmentos . . . Depois, Path.resolve() elimina aliases e segue links simbólicos; relative_to(root) garante que o resultado permanece sob a raiz. Um alvo seguro inexistente resulta em 404; uma tentativa de sair da raiz resulta em 403. A extensão determina o MIME; extensões desconhecidas usam application/octet-stream.

Tentativa enviada com curl --path-as-is	Resposta esperada	Evidência real
../segredo.txt	403	PENDENTE
/%2e%2e/segredo.txt	403	PENDENTE
/%5csegredo.txt	403	PENDENTE

As três tentativas incluem percurso direto, percent-encoding e barra invertida. O relatório final deve mostrar cada linha GET observada e sua resposta 403 na rede real.

5. Conformidade dos métodos, respostas e arquivos

GET envia cabeçalhos e corpo. HEAD calcula o mesmo arquivo, tipo e comprimento do GET, mas não envia corpo. Outro método recebe 405 e Allow: GET, HEAD. O servidor produz 200, 400, 403, 404 e 405. Para evitar que bytes de um corpo de requisição sejam lidos como outro request na conexão persistente, GET/HEAD com corpo ou Transfer-Encoding são rejeitados com 400 e fechamento. Métodos não suportados recebem 405 e fechamento.

Status	Requisição de teste	Conteúdo a registrar na captura
200	curl -i http://IP:8080/	HTTP/1.1 200, tamanho, tipo HTML, corpo
400	curl -i --request-target "" http://IP:8080/	HTTP/1.1 400, corpo de erro e comprimento
403	curl -i --path-as-is http://IP:8080../segredo.txt	HTTP/1.1 403; arquivo externo não servido
404	curl -i http://IP:8080/nao-existe.txt	HTTP/1.1 404
405	curl -i -X POST http://IP:8080/	HTTP/1.1 405 e Allow: GET, HEAD

Resultados entre máquinas: PENDENTE. Os testes automatizados cobrem esses casos, HEAD, MIME obrigatório (.html, .css, .js, .json, .txt, .png, .jpg, .pdf) e fallback binário. A página em www/ referencia apenas o JPEG para que o navegador faça requisições separadas para HTML e imagem.

6. Conexões persistentes e timeout

HTTP/1.1 mantém a conexão aberta por padrão. O servidor declara Connection: keep-alive e volta a ler o mesmo socket após cada resposta. Connection: close é tratado como token sem distinção de maiúsculas; o servidor responde Connection: close e encerra após enviar a resposta. Uma conexão ociosa por 5 segundos é fechada sem enviar uma resposta extra. Uma desconexão limpa do cliente também encerra normalmente.

O comprimento explícito de todas as respostas permite ao cliente localizar o fim de cada corpo sem depender do fechamento TCP. O cliente C2 verifica dez respostas completas em sequência; a décima inclui `Connection: close`. Os testes automatizados cobrem sequência, requisições agrupadas num `recv`, fechamento explícito e `timeout`.
Evidência Wireshark da conexão única: PENDENTE.

7. Captura de uma transação completa

Capturar um GET bem-sucedido feito da máquina cliente e identificar:

Etapa	Número do pacote e observação
SYN do cliente	PENDENTE
SYN-ACK do servidor	PENDENTE
ACK que completa o handshake	PENDENTE
Pacote com GET	PENDENTE
Pacotes com resposta HTTP	PENDENTE
FIN/ACK e encerramento	PENDENTE

Usar um GET com `Connection: close` facilita mostrar todo o ciclo. Incluir imagem da captura e arquivo `.pcapng` original na entrega.

8. Experimento C1 x C2 e tabela comparativa

O cliente de medição faz dez GETs sequenciais do **mesmo** recurso `/leia-me.txt`, entre máquinas distintas, com Wireshark ativo. C1 abre uma conexão por GET e solicita `Connection: close` em cada um. C2 reutiliza uma conexão para os dez GETs e solicita fechamento apenas no último. Capturar separadamente `c1.pcapng` e `c2.pcapng`, usando o mesmo filtro, recurso, porta, máquinas e critério de bytes.

RTT médio por ping: PENDENTE ms. Cenário de rede e tamanho do arquivo: PENDENTE.

Métrica extraída da captura	C1	C2
Handshakes TCP completos	PENDENTE (esperado 10)	PENDENTE (esperado 1)
Pacotes totais	PENDENTE	PENDENTE
Bytes totais (<code>frame.len</code>)	PENDENTE	PENDENTE
Tempo total, primeiro SYN ao último pacote (s)	PENDENTE	PENDENTE
Tempo do cliente (s), referência separada	PENDENTE	PENDENTE

Economia de pacotes (%) = $(\text{pacotes_C1} - \text{pacotes_C2}) / \text{pacotes_C1} * 100$. Economia de bytes (%) = $(\text{bytes_C1} - \text{bytes_C2}) / \text{bytes_C1} * 100$. Calcular as porcentagens só após preencher as capturas:
PENDENTE.

9. Overhead de conexão e interpretação do RTT

Em C1, marcar em cada um dos dez fluxos os pacotes exclusivamente de abertura (SYN, SYN-ACK, ACK final) e encerramento (FIN/ACK correspondentes). Somar a quantidade de pacotes e os bytes `frame.len` desses quadros. ACKs que confirmam dados não devem ser classificados indiscriminadamente como overhead puro.

Overhead C1 em pacotes: PENDENTE. Overhead C1 em bytes: PENDENTE.

C1 abre dez conexões e C2 abre uma: são **nove aberturas extras**. O handshake clássico consome aproximadamente um RTT antes de a primeira requisição de cada conexão poder ser atendida. Portanto, a previsão é de aproximadamente $9 \times \text{RTT_médio}$ de custo extra de abertura em C1. A diferença observada $\text{tempo_C1} - \text{tempo_C2}$ será **PENDENTE** s. Ela não precisa ser exatamente $9 \times \text{RTT}$: tamanho dos pacotes, encerramento, agendamento das threads, retransmissões e variação de rede também influenciam.

Quanto maior o RTT entre cliente e servidor, maior a economia temporal potencial por evitar essas nove aberturas. Quanto mais recursos sequenciais forem requisitados, maior a parcela de handshakes evitados. Essa conclusão decorre da contagem de conexões e deve ser confrontada com as capturas.

10. Interoperabilidade, conclusão e entrega

O navegador de outro grupo deve abrir `www/index.html` no nosso servidor; nosso navegador deve abrir a página do servidor deles. O teste passa quando HTML e JPEG carregam. **Resultado presencial: PENDENTE.** Todos os integrantes devem conseguir explicar sockets, parser, respostas, segurança, threads, persistência e análise C1/C2.

O código implementa os requisitos funcionais das duas partes e passou os testes locais automatizados. A conclusão quantitativa sobre pacotes, bytes e tempo só será fechada com as capturas entre máquinas distintas. A entrega final é um único `.zip` ou `.tar` contendo fontes, `README.md`, `www/`, `capturas/`, `*.pcapng` e este relatório em PDF, sem binários ou caches.

Apêndice A. Verificação reproduzível e portabilidade

Execute `py -3 -m unittest discover -s tests -v` no Windows ou `python3 -m unittest discover -s tests -v` no macOS/Linux. Os testes unitários verificam fragmentação e agrupamento de bytes, parser, cabeçalhos, MIME e proteção de caminho; os de integração abrem sockets reais em `127.0.0.1` para GET, HEAD, erros, dez requisições na mesma conexão, fechamento, ociosidade, clientes simultâneos e os clientes de medição C1/C2. A porta dos testes é atribuída pelo sistema, evitando conflito com 8080. A matriz de CI em `.github/workflows/compatibility.yml` roda a suíte em Windows e Linux com Python 3.9 e 3.13 e em macOS com 3.13. A biblioteca padrão é a única dependência de execução do servidor; ReportLab é opcional e usado apenas para regerar este PDF. Testes locais não substituem as capturas reais nem provam que firewall e rede da VDI permitem conexões de entrada.

Apêndice B. Escopo e limitações conhecidas

Este servidor didático usa IPv4 e arquivos estáticos. Não implementa HTTPS, HTTP/2, corpo de requisição, transferência em chunks, Range, compressão, cache condicional ou autenticação. Ele lê cada arquivo inteiro na memória e cria uma thread por conexão; portanto não é adequado para grandes arquivos ou alta escala. O limite de cabeçalhos é 64 KiB e o timeout ocioso é 5 s. Somente o formato de alvo `/caminho` e HTTP/1.1 são aceitos. Essas restrições são intencionais e não impedem os objetivos especificados no trabalho.

Apêndice C. Matriz de conferência antes da entrega

Caso	Como verificar	Critério de aprovação
Inicialização	CLI com raiz existente e porta 8080	Log mostra escuta em <code>0.0.0.0:8080</code>
Página e recursos	Navegador remoto e aba de rede	HTML e JPEG com 200
GET e HEAD	<code>curl -i /</code> e <code>curl -I /</code>	Mesmo tipo/tamanho; HEAD sem corpo
Erros	Comandos da seção 5	400, 403, 404 e 405 corretos
Cabeçalhos	Inspeção com curl/Wireshark	Date, Server, Length, Type, Connection
Segurança	Três comandos <code>--path-as-is</code>	Três 403, nenhum arquivo externo
Paralelismo	Cliente lento B e GET rápido C	C termina enquanto B espera
Persistência	Cliente C2 e captura	Dez respostas, um handshake
Fechamento	Connection: close e <code>idle_client.py</code>	Close explícito e timeout ~5 s
C1/C2	Capturas separadas na rede real	10/1 handshakes e métricas preenchidas
Interoperabilidade	Navegadores de dois grupos	Recursos de ambos carregam
Portabilidade	Suíte na VDI e colegas	OK no Python 3.9+ em cada SO

O roteiro `ROTEIRO_DE_TESTES.md` contém os comandos completos para cada plataforma. Antes de enviar o arquivo final, preencher os itens PENDENTE, inserir as figuras e os `.pcapng`, confirmar a exatidão dos números e testar o pacote extraído numa máquina diferente. Sem essas evidências, o software está implementado, mas o relatório experimental ainda não está concluído.

Referências

- IETF, RFC 9110, HTTP Semantics: <https://www.rfc-editor.org/rfc/rfc9110.html>
- IETF, RFC 9112, HTTP/1.1: <https://www.rfc-editor.org/rfc/rfc9112.html>
- Wireshark Foundation, User's Guide: https://www.wireshark.org/docs/wsug_html/
- Kurose e Ross, Redes de Computadores e a Internet, seção 2.2.